

Parámetros



◆ top_p (nucleus sampling)

Define el porcentaje acumulado de probabilidad de palabras entre las que el modelo puede elegir.

👉 Cómo funciona:

1. El modelo ordena todas las posibles siguientes palabras por probabilidad.
2. Suma probabilidades hasta llegar al límite p .
3. Solo elige entre ese conjunto reducido.

Ejemplo:

Si $\text{top_p} = 0.7$, solo se considerarán las palabras cuya probabilidad acumulada llegue al 70%.

Idea clave: controla la diversidad limitando el "universo" de palabras posibles por probabilidad acumulada.

Parámetros



Ejemplo práctico de *top_p*

Supongamos que el modelo quiere predecir la siguiente palabra después de:

"El cielo está"

El modelo calcula probabilidades (ejemplo inventado para explicarlo):

Palabra	Probabilidad
azul	0.40
nublado	0.30
despejado	0.15
oscuro	0.10
rojo	0.05

◆ Caso 1: *top_p* = 0.9

Queremos las palabras cuya **probabilidad acumulada** ≤ 0.9 .

1. azul (0.40) → acumulado: 0.40
2. nublado (0.30) → acumulado: 0.70
3. despejado (0.15) → acumulado: 0.85
4. oscuro (0.10) → acumulado: 0.95 ❌ se pasa de 0.90 → se excluye

◆ Palabras permitidas = { azul, nublado, despejado }

Resultado: el modelo elegirá una de esas 3, normalmente azul o nublado.

◆ Caso 2: *top_p* = 0.5

Corte mucho más estricto:

1. azul (0.40) → acumulado: 0.40
2. nublado (0.30) → acumulado: 0.70 ❌ se pasa del 0.50

◆ Palabras permitidas = { azul }

Resultado: siempre "azul" → **respuesta más determinista**.

Parámetros



◆ Caso 3: top_p = 1.0

Incluye **todas** las palabras.

- ◆ **Palabras permitidas = todas**

Resultado: más variabilidad → incluso podría salir "rojo".

En resumen visual

top_p	Qué pasa	Efecto
0.3 – 0.5	Muy pocas palabras permitidas	Respuestas más repetitivas, "seguras"
0.7 – 0.9	Equilibrio entre control y variedad	Respuestas naturales
1.0	Todas las palabras posibles	Más creatividad, más riesgo

Parámetros



◆ top_k (k-best sampling)

Limita el número de palabras posibles a las **k más probables**, sin mirar porcentajes.

👉 Cómo funciona:

1. El modelo selecciona las **k** palabras más probables.
2. **Solo elige entre esas** (por ejemplo, las 40 más probables si `top_k = 40`).

Idea clave: controla la diversidad fijando un **número máximo de alternativas**.

Parámetros



Ejemplo práctico de *top_k*

Imagina que el modelo intenta predecir la siguiente palabra después de:

"El cielo está"

El modelo calcula estas probabilidades (ejemplo ilustrativo):

Palabra	Probabilidad
azul	0.40
nublado	0.30
despejado	0.15
oscuro	0.10
rojo	0.05

◆ Caso 1: *top_k* = 3

El modelo solo puede elegir entre las 3 palabras más probables.

1. azul
2. nublado
3. despejado

◆ Palabras permitidas = { azul, nublado, despejado }

Resultado:

Respuesta variada, pero dentro de las opciones más probables.

◆ Caso 2: *top_k* = 1

Solo se toma la palabra más probable:

1. azul

◆ Palabras permitidas = { azul }

Resultado:

Siempre responderá "azul".

👉 Respuesta totalmente determinista.

Parámetros



◆ Caso 3: $\text{top_k} = 5$ (o un valor $\geq n^\circ$ total de palabras posibles)

Se permiten todas:

- ◆ Palabras permitidas = todas

Resultado:

Generación más creativa y diversa.

Resumen rápido

top_k	Qué hace	Efecto
1	Solo la palabra más probable	Respuesta fija
3–10	Palabras más probables	Equilibrio entre precisión y variedad
Mucho más alto	Casi todas las palabras	Más creatividad, menos control

Parámetros



Ejemplo práctico

Con esta config:

```
1 top_p = 0.95
2 top_k = 40
3 max_output_tokens = 100
```

    `</> Python`

En cada palabra del texto:

- El modelo considera **hasta 40** alternativas (por top_k).
- De esas, se queda solo con las que suman **95% de probabilidad** (top_p).
- Elige una.

👉 Repite esto hasta llegar a 100 tokens → y ahí corta.